Instituto Tecnológico Metropolitano (ITM)

Mini-Lakehouse TuBoleta

Análisis de la venta de boletería con arquitectura Bronze / Silver / Gold, Parquet y DuckDB

Informe final

Curso: Tópicos Avanzados en Bases de Datos — Taller 3

Integrantes
Luis Manuel Contreras
Juan Pablo Salazar

Docente
Roberto Carlos Rahamut Suteu

20 de octubre de 2026

Repositorio: github.com/JPS4L4/tuboleta_lakehouse

1. Dominio y proceso analizado

TuBoleta vende boletería masiva para el Estadio Atanasio Girardot (Medellín). Su sistema operacional combina dos almacenes con funciones distintas:

- **Redis (AP):** fila virtual, locks temporales de silla con TTL de 600 s y contadores. Es estado **efímero**.
- **CockroachDB (CP):** fuente de verdad. Guarda las compras confirmadas y evita el *double-booking* con `UNIQUE(evento, silla)` bajo `SERIALIZABLE`.

El flujo de una compra es: entrar a la fila, ser admitido, bloquear una silla (lock con TTL) y pagar a tiempo (`INSERT` en CockroachDB) o dejar vencer el TTL (abandono; la silla se libera). Si otro usuario ya compró la silla, el `UNIQUE` rechaza el `INSERT`.

Se analizan dos procesos:

1. **Ventas:** cuántas boletas y cuánto dinero se vendió por evento, zona, canal y período. Una fila es una silla vendida y pagada.
2. **Embudo de compra:** cómo terminan los intentos (compra, abandono o rechazo `UNIQUE`). Una fila es un intento.

Los datos son **sintéticos y deterministas** (semilla 42), con cinco eventos, cuatro zonas, tres canales, 45.000 sillas y 150.000 usuarios. Las tasas de conversión, abandono y descuento son parámetros del generador, no datos reales.

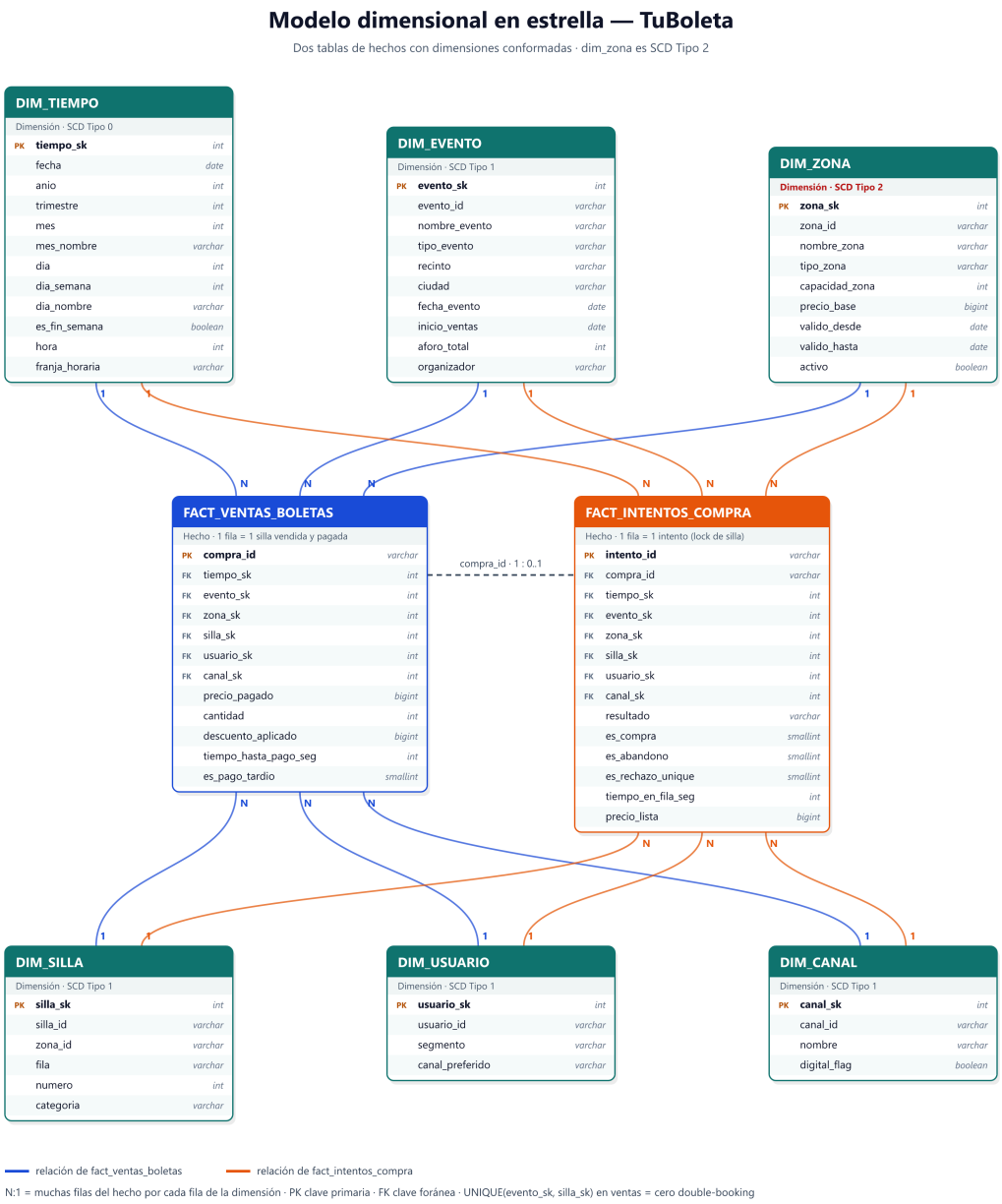
2. Arquitectura del Lakehouse

```
OLTP (CockroachDB + Redis)         datos sintéticos (por defecto)
          |                                  |
          +----------------+-----------------+
                           v
         BRONZE  lakehouse/bronze   Parquet crudo y sucio, versionado
                           |  transform_silver.py (limpieza técnica)
                           v
         SILVER  lakehouse/silver   Parquet limpio y tipado
                           |  transform_gold.py (estrella + SCD2)
                           v
         GOLD    lakehouse/gold     2 hechos + 6 dimensiones
                           |  DuckDB (read_parquet)
                           v
         analyze.py / consultar.py  consultas + benchmark -> docs/
```

Capa	Contenido	Responsabilidad
Bronze	Parquet tal como llega, con suciedad intencional	Conservar el dato crudo, un archivo por extracción
Silver	Parquet limpio y tipado	Solo limpieza técnica: deduplicar, `strip`, convertir fechas, resolver nulos, descartar filas sin clave
Gold	Estrella: 2 hechos y 6 dimensiones	Modelar para análisis y resolver claves sustitutas
DuckDB	Vistas `read_parquet` sobre Gold	Consultas analíticas sin cargar datos a un motor

`main.py` orquesta extracción, Silver, Gold y análisis en un solo comando y tarda unos 8 segundos. Los Parquet no se versionan: se regeneran en cada ejecución.

3. Diseño dimensional



Modelo dimensional en estrella

El modelo fuente está en `docs/modelo_dimensional.dbml`; la imagen se generó a partir de ese DBML.

Las cinco preguntas

1. Proceso de negocio. El comportamiento de las ventas de boletas por evento, zona, canal y período, junto con el embudo de intentos que lo precede.

2. Granularidad.

Tabla de hechos	Una fila es...
`fact_ventas_boletas`	una silla vendida y pagada en un evento
`fact_intentos_compra`	un intento de compra (lock de una silla) que termina en `COMPRA`, `ABANDONO` o `RECHAZO_UNIQUE`

Los abandonos no son ventas: si se mezclaran, cada `SUM(precio_pagado)` tendría que acordarse de filtrar. El pipeline verifica con `assert` que no hay *double-booking*, que cada venta enlaza con exactamente un intento `COMPRA` y que cada hecho resolvió exactamente una versión de zona.

3. Dimensiones (conformadas). Ambos hechos comparten las seis dimensiones.

Dimensión	Tipo SCD	Nota
`dim_tiempo`	Tipo 0	Granularidad de hora; franjas madrugada, mañana, tarde y noche
`dim_evento`	Tipo 1	Nombre, fecha, inicio de ventas, aforo
`dim_zona`	**Tipo 2**	Única cuyos cambios alteran el significado de las métricas (el precio)
`dim_silla`	Tipo 1	Zona, fila, número
`dim_usuario`	Tipo 1	Segmento y canal preferido
`dim_canal`	Tipo 1	Incluye el miembro `DESCONOCIDO` para canales nulos

4. Métricas.

Métrica	Hecho	Tipo
`precio_pagado`, `descuento_aplicado`, `cantidad`	ventas	Aditiva
`tiempo_hasta_pago_seg`	ventas	No aditiva (solo `AVG` o percentiles)
`es_pago_tardio`	ventas	Flag: pago en el último 20 % del TTL; sigue siendo venta válida
`es_compra`, `es_abandono`, `es_rechazo_unique`	intentos	Flags 0/1 para conteos y tasas
`tiempo_en_fila_seg`	intentos	No aditiva
`precio_lista`	intentos	Aditiva; con abandono estima el ingreso potencial perdido

Con esta granularidad no hay métricas semi-aditivas. Aparecerían con una instantánea periódica de inventario (por ejemplo `sillas_disponibles` por zona y día): se puede sumar entre zonas, pero no a lo largo del tiempo, donde se usa el último valor o el promedio del período.

5. Cambios lentos (SCD Tipo 2 en `dim_zona`). Cuando cambia el precio de una zona se cierra la versión anterior (`valido_hasta`, `activo = false`) y se inserta una nueva con otro `zona_sk`. Cada hecho guarda el `zona_sk` vigente en su fecha. El 2026-07-01 suben VIP y Occidental:

zona_sk	zona_id	nombre_zona	precio_base	valido_desde	valido_hasta	activo
1	Z-VIP	VIP	250.000	2026-01-01	2026-06-30	false
2	Z-VIP	VIP	280.000	2026-07-01	9999-12-31	true
3	Z-OCC	Occidental	150.000	2026-01-01	2026-06-30	false
4	Z-OCC	Occidental	165.000	2026-07-01	9999-12-31	true

Hay eventos cuya ventana de ventas cruza el 1 de julio, así que un mismo evento vende boletas a dos precios. Los análisis por zona agrupan por `zona_id`, nunca por versión, y no filtran `activo = true`: de lo contrario se perderían las ventas anteriores al cambio.

El estado de Redis no es un hecho. La fila virtual, los locks vivos y los contadores son efímeros y no describen un evento terminado. Sí es hecho el resultado de cada intento que el sistema operacional persiste.

4. Pipeline ETL y reporte de calidad

Extracción. Genera datos sintéticos (o extrae de CockroachDB con `USE_SYNTHETIC=false`) hacia Bronze, un archivo Parquet por tabla y por extracción.

Suciedad intencional en Bronze. Para que Silver tenga trabajo real, el generador inyecta los problemas típicos de un volcado operacional: filas duplicadas (~1 %), IDs con espacios sobrantes (~1,5 %), fechas como texto, `canal_id` nulo (~1,5 %) y 40 filas sin clave de negocio.

Silver deduplica por clave de negocio, aplica `strip`, convierte fechas, reemplaza el canal nulo por `DESCONOCIDO` y descarta las filas sin clave. **Gold** construye las dimensiones, resuelve claves sustitutas y la versión de zona, y ejecuta las verificaciones de integridad.

Reporte de calidad de la última ejecución (`docs/reporte_calidad_silver.txt`):

Tabla	Filas Bronze	Filas Silver	% retenido	Duplicados	Sin clave	Nulos resueltos
canales	3	3	100,00	0	0	0
zonas	6	6	100,00	0	0	0
eventos	5	5	100,00	0	0	0
sillas	45.000	45.000	100,00	0	0	0
usuarios	150.750	150.000	99,50	750	0	0
compras_confirmadas	210.358	208.236	98,99	2.082	40	3.101
intentos_compra	262.936	260.294	99,00	2.602	40	3.906

En `compras_confirmadas` la suma de `precio_pagado` baja de 22.352.656.600 COP en Bronze a 22.126.123.800 COP en Silver: la diferencia son los duplicados y las filas sin clave, que inflaban los ingresos.

Volúmenes de Gold.

Tabla	Filas
`fact_ventas_boletas`	208.236
`fact_intentos_compra`	260.294 (+25 % sobre las ventas)
Sillas / usuarios	45.000 / 150.000
Eventos / versiones de zona	5 / 6
`dim_tiempo` (hora)	5.656

5. Consultas analíticas

Se ejecutan nueve consultas de negocio sobre Gold con DuckDB (`scripts/analyze.py`); las salidas completas están en `docs/resultados_consultas.md` . Los ingresos suman **22.126 millones de COP con 208.236 boletas**. Resultados clave:

Ingresos por canal. La app vende el 44 %, la web el 39 %, la taquilla el 15 % y el 1,5 % no tiene canal registrado.

Canal	Boletas	%	Ingresos (COP)	Ticket promedio
app	92.066	44,21	9.779.126.100	106.219
web	82.058	39,41	8.716.875.500	106.228
taquilla	31.011	14,89	3.301.901.100	106.475
DESCONOCIDO	3.101	1,49	328.221.100	105.844

Ocupación. Entre 85,8 % y 100 % por zona; el Clásico Paisa agotó VIP y Occidental.

Embudo de conversión por evento. De cada 100 intentos, cerca de 80 terminan en compra, 14 en abandono y 6 en rechazo `UNIQUE` ; alrededor del 8 % de las ventas se paga en el último tramo del TTL (estas tasas salen de los parámetros del generador).

Evento	Intentos	Compras	Abandonos	Rechazos	Conversión %
Clásico Paisa Nacional vs Medellín	54.577	43.625	7.468	3.484	79,93
Festival Sonidos del Valle	53.295	42.455	7.373	3.467	79,66
Final del Fútbol Colombiano	53.161	42.667	7.174	3.320	80,26
Gira Urbana Medellín	49.947	40.069	6.752	3.126	80,22
Concierto Noche Latina	49.314	39.420	6.633	3.261	79,94

SCD Tipo 2: VIP por versión de precio. Sin SCD2 esta diferencia de precio se perdería.

zona_sk	Precio base	Vigencia	Boletas	Ingresos (COP)	Ticket promedio
1	250.000	2026-01-01 a 2026-06-30	5.834	1.411.662.500	241.972
2	280.000	desde 2026-07-01	8.041	2.186.856.000	271.963

Horarios. Tarde (35,85 %) y noche (34,66 %) concentran cerca del 70 % de las ventas; el primer día de venta de cada evento concentra alrededor del 10 %.

Abandono. Los abandonos asocian 3.868 millones de COP de precio de lista, una cota superior del ingreso potencial perdido (parte de esas sillas se vendió después a otro usuario). Por zona: Occidental 1.494, Norte 994, General 757 y VIP 623 millones de COP.

6. Benchmark OLTP vs columnar

Misma agregación lógica en tres motores: boletas e ingresos por **evento x zona x mes** (60 grupos, 208.236 ventas). No se mide la carga; se hace un calentamiento y se toma la mediana de 5 corridas. Los tres motores devuelven exactamente las mismas cifras (verificado con una aserción). Máquina: Windows 11, 12 hilos lógicos, Python 3.12.10, DuckDB 1.5.6, SQLite 3.49.1.

Motor	Fuente	Mediana (ms)	Rango (ms)	Mejora vs (a)
(a) SQLite, almacén por filas (OLTP-like)	Silver normalizado + índices típicos	368,89	354,60 - 386,53	1,0x
(b) DuckDB sobre Bronze crudo	Parquet sucio, limpieza en la consulta	67,14	59,98 - 71,66	5,5x
(c) DuckDB sobre Gold (estrella)	Parquet columnar modelado	17,57	14,21 - 19,23	21,0x

Análisis. DuckDB sobre Gold fue unas 21 veces más rápido que el almacén por filas. Buena parte de la ventaja ya aparece solo por leer Parquet columnar sin modelar (5,5x en Bronze); el modelado en estrella aporta otro factor de casi 4 sobre Bronze, porque evita limpiar al vuelo y trabaja con claves enteras y dimensiones pequeñas.

Salvedades honestas.

- La comparación no es perfectamente equivalente: DuckDB usa varios hilos (12) y SQLite solo uno, así que parte de la diferencia no viene del formato.
- SQLite corre con índices pensados para buscar filas, no para recorrer 208.000 ventas.
- Con este volumen todas las consultas responden en menos de medio segundo; la diferencia se vuelve decisiva a millones de filas y no cambia el resultado de negocio. Con menos de 100.000 filas las diferencias se reducen.
- Los tiempos varían unos milisegundos entre ejecuciones.
- El almacén por filas del benchmark es SQLite, no una instancia real de CockroachDB.

7. Limitaciones y evolución futura

- **Datos sintéticos.** Los patrones son plausibles pero no reales; las tasas son parámetros del generador.
- **Carga completa.** Cada ejecución reconstruye Silver y Gold. A mayor volumen habría que pasar a cargas incrementales (por rango de fechas o marca de agua).
- **Sin captura de cambios.** La extracción es por lote. Una evolución natural es CDC con Kafka (Debezium sobre los *changefeeds* de CockroachDB) para alimentar Bronze casi en tiempo real.
- **CockroachDB real.** La rama de extracción existe (`USE_SYNTHETIC=false`) pero no se probó contra un clúster.
- **SCD Tipo 2 en `dim_usuario`.** Hoy es Tipo 1; historizar segmento y canal preferido permitiría analizar cómo cambia un mismo cliente.
- **Instantánea de inventario.** Una tabla periódica de `sillas_disponibles` por zona y día introduciría una métrica semi-aditiva.
- **Embudo parcial.** Empieza en el lock de la silla; las salidas de la fila antes de lograr un lock no se persisten y no se analizan.
- **Una compra es una silla.** No se modela el carrito de varias sillas en una transacción.

8. Cómo reproducir

Requisitos: Python 3.10 o superior (probado con 3.12). No se necesita CockroachDB.

```
git clone https://github.com/JPS4L4/tuboleta_lakehouse.git
cd tuboleta_lakehouse
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python main.py
```

`main.py` corre extracción, Silver, Gold y análisis, imprime el tiempo de cada paso y termina con código 1 si alguno falla. Cada paso también se puede correr solo, por ejemplo `python -m scripts.transform_silver`.

Consultas en vivo sobre Gold, con el tiempo en milisegundos:

```
python -m scripts.consultar --lista
python -m scripts.consultar ocupacion
python -m scripts.consultar "SELECT COUNT(*) AS boletas FROM fact_ventas_boletas"
```

Archivo	Contenido
`README.md`	Guía de uso, decisiones y resultados
`docs/diseno_dimensional.md`	Las cinco preguntas, SCD2 y limitaciones
`docs/modelo_dimensional.dbml` y `.png`	Modelo en estrella
`docs/reporte_calidad_silver.txt`	Bronze vs Silver
`docs/resultados_consultas.md`	Salida de las nueve consultas
`docs/benchmark.md`	Tabla y condiciones del benchmark